

ASSIGNMENT-9

NAME: SRAVANI ANUMALA

ROLL NO: 2303A510G7

BATCH-30

CODE:

```
class LendingContract:
```

```
    def __init__(self):
```

```
        self.balances = {}
```

```
    def deposit(self, user, amount):
```

```
        if user in self.balances:
```

```
            self.balances[user] += amount
```

```
        else:
```

```
            self.balances[user] = amount
```

```
        print(user, "deposited", amount, "ETH")
```

```
    def withdraw(self, user, amount):
```

```
        if user not in self.balances:
```

```
            print("User not found")
```

```
        elif self.balances[user] < amount:
```

```
            print("Insufficient balance")
```

```
        else:
```

```
self.balances[user] -= amount  
print(user, "withdrew", amount, "ETH")
```

```
def show_balance(self, user):  
    print(user, "balance:", self.balances.get(user, 0), "ETH")
```

Example usage

```
contract = LendingContract()
```

```
contract.deposit("Alice", 5)
```

```
contract.deposit("Bob", 3)
```

```
contract.show_balance("Alice")
```

```
contract.show_balance("Bob")
```

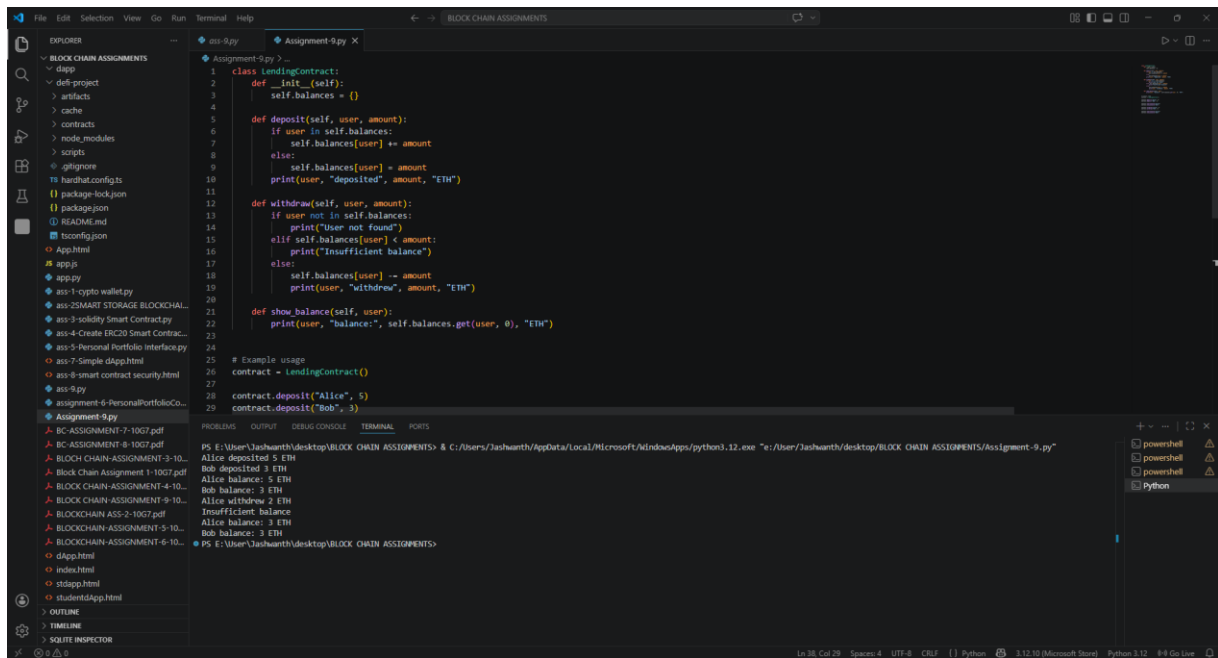
```
contract.withdraw("Alice", 2)
```

```
contract.withdraw("Bob", 5)
```

```
contract.show_balance("Alice")
```

```
contract.show_balance("Bob")
```

OUTPUT:



The image shows a Visual Studio Code editor window with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer shows a project named 'BLOCK CHAIN ASSIGNMENTS' with various files and folders. The code editor displays a Python script named 'Assignment-9.py' which defines a 'LendingContract' class. The script includes methods for depositing, withdrawing, and showing balances. The terminal shows the output of running the script, which demonstrates the contract's functionality by depositing and withdrawing funds and checking balances.

```
1 class LendingContract:
2     def __init__(self):
3         self.balances = {}
4
5     def deposit(self, user, amount):
6         if user in self.balances:
7             self.balances[user] += amount
8         else:
9             self.balances[user] = amount
10        print(user, "deposited", amount, "ETH")
11
12    def withdraw(self, user, amount):
13        if user not in self.balances:
14            print("User not found")
15        elif self.balances[user] < amount:
16            print("Insufficient balance")
17        else:
18            self.balances[user] -= amount
19            print(user, "withdrew", amount, "ETH")
20
21    def show_balance(self, user):
22        print(user, "balance:", self.balances.get(user, 0), "ETH")
23
24
25 # Example usage
26 contract = LendingContract()
27
28 contract.deposit("Alice", 5)
29 contract.deposit("Bob", 3)
```

PS E:\User\Jashwanth\Desktop\BLOCK CHAIN ASSIGNMENTS> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "e:/user/jashwanth/desktop/BLOCK CHAIN ASSIGNMENTS/Assignment-9.py"

Alice deposited 5 ETH
Bob deposited 3 ETH
Alice balance: 5 ETH
Bob balance: 3 ETH
Alice withdrew 2 ETH
Insufficient balance
Alice balance: 3 ETH
Bob balance: 3 ETH

PS E:\User\Jashwanth\Desktop\BLOCK CHAIN ASSIGNMENTS>